

Basic Codes – Mathematics (DSA Notes)

1. Count Digits

Algorithm:

1. If number is 0, answer is 1.
2. Initialize count = 0.
3. While number > 0, increment count and divide number by 10.

Java Code:

```
class Solution {
    public int countDigits(int n) {
        if (n == 0) return 1;
        int count = 0;
        while (n > 0) {
            count++;
            n /= 10;
        }
        return count;
    }
}
```

2. Reverse a Number

Algorithm:

1. Initialize rev = 0.
2. Extract last digit using % 10.
3. Update rev = rev * 10 + digit.
4. Remove last digit using / 10.

Java Code:

```
class Solution {
    public int reverseNumber(int n) {
        int rev = 0;
        while (n > 0) {
            int digit = n % 10;
            rev = rev * 10 + digit;
            n /= 10;
        }
        return rev;
    }
}
```

3. Check Palindrome

Algorithm:

1. Store original number.
2. Reverse the number.
3. Compare reversed with original.

Java Code:

```
class Solution {
    public boolean isPalindrome(int n) {
        int original = n;
        int rev = 0;
        while (n > 0) {
            rev = rev * 10 + (n % 10);
            n /= 10;
        }
        return rev == original;
    }
}
```

```
}
```

4. GCD / HCF

Algorithm (Euclidean):

1. While $b \neq 0$:

$a = b, b = a \% b$

2. a is the GCD.

Java Code:

```
class Solution {  
    public int gcd(int a, int b) {  
        while (b != 0) {  
            int temp = b;  
            b = a % b;  
            a = temp;  
        }  
        return a;  
    }  
}
```

5. Armstrong Number

Algorithm:

1. Count number of digits.

2. Extract each digit and raise to power digits.

3. Add to sum.

4. Compare sum with original number.

Java Code:

```
class Solution {  
    public boolean isArmstrong(int n) {  
        int original = n;  
        int digits = String.valueOf(n).length();  
        int sum = 0;  
        while (n > 0) {  
            int digit = n % 10;  
            sum += Math.pow(digit, digits);  
            n /= 10;  
        }  
        return sum == original;  
    }  
}
```

6. Print All Divisors

Algorithm:

1. Loop from 1 to $\sqrt{n}$.

2. If i divides n , print i and n/i .

Java Code:

```
class Solution {  
    public void printDivisors(int n) {  
        for (int i = 1; i * i <= n; i++) {  
            if (n % i == 0) {  
                System.out.print(i + " ");  
                if (i != n / i) {  
                    System.out.print((n / i) + " ");  
                }  
            }  
        }  
    }  
}
```

7. Check Prime

Algorithm:

1. If $n \leq 1$, not prime.
2. Check divisibility till $\sqrt{n}$.

Java Code:

```
class Solution {  
    public boolean isPrime(int n) {  
        if (n <= 1) return false;  
        for (int i = 2; i * i <= n; i++) {  
            if (n % i == 0) return false;  
        }  
        return true;  
    }  
}
```